

Gaussian null study: convergence rate of the false-positive probability

J. Dedecker, M.-L. Taupin, V. Runge and A.-S. Tocquet

2026-07-01

Contents

1	Objective	1
2	Important Monte Carlo point	2
3	Parameters	2
4	Original model-free screening test	3
5	One Gaussian-null data set	4
6	Estimate the FPR for one sample size	4
7	Monte Carlo study	5
8	FPR convergence-rate table	6
9	Log-log rate estimate	6
10	Log-log plot	7
11	Interpretation	8
12	Save results	8

1 Objective

This study considers the simplest null setting:

$$X_i \sim \mathcal{N}(0, 1), \quad Y_i \sim \mathcal{N}(0, 1), \quad X_i \perp Y_i.$$

Each simulated data set contains one Gaussian predictor column X and one independent Gaussian response Y . Repeating the experiment over many independent data sets is equivalent to studying many independent null columns.

For a sample size n , define the finite-sample false-positive probability

$$\text{FPR}_n = \mathbb{P} \{ |\hat{\gamma}_n| \geq \Phi^{-1}(1 - q/2) \}.$$

The quantity expected to decrease at rate $C/\sqrt{n}$ is the finite-sample bias relative to the nominal level:

$$|\text{FPR}_n - q|.$$

Therefore, the target slope in the log-log regression of $|\widehat{\text{FPR}}_n - q|$ against n is $-1/2$.

2 Important Monte Carlo point

With a fixed number of simulations B , the Monte Carlo error of $\widehat{\text{FPR}}_n$ is of order $1/\sqrt{B}$, independently of n . It will eventually mask a true $1/\sqrt{n}$ bias.

For this reason, this document uses a number of simulated null columns B_n proportional to n . The resulting Monte Carlo standard error is also proportional to $1/\sqrt{n}$, but is chosen to be a small fraction of the target rate.

3 Parameters

```
knitr::opts_chunk$set(
  echo = TRUE,
  eval = TRUE,
  cache.path = "gaussian_null_fpr_rate_cache/"
)

library(parallel)
library(knitr)

# Nominal two-sided false-positive rate.
q <- 0.50

# Sample sizes for the log-log rate study.
n_values <- round(1.5^(2:9) * 200)

# The desired Monte Carlo standard error is approximately
# mc_error_fraction / sqrt(n).
mc_error_fraction <- 0.15

# For a Bernoulli(q) selection indicator,
# sqrt(q * (1 - q) / B_n) <= mc_error_fraction / sqrt(n)
# is obtained with B_n >= q(1-q)n / mc_error_fraction^2.
B_by_n <- ceiling(q * (1 - q) * n_values / mc_error_fraction^2)
```

```

# For a quick test, use the following instead:
# B_by_n <- rep(2000L, length(n_values))

base_seed <- 20260701L
n_cores <- min(8L, parallel::detectCores(logical = FALSE))

data.frame(n = n_values, B_n = B_by_n)

##      n   B_n
## 1  450  5000
## 2  675  7500
## 3 1012 11245
## 4 1519 16878
## 5 2278 25312
## 6 3417 37967
## 7 5126 56956
## 8 7689 85434

```

4 Original model-free screening test

```

screening_test_ML <- function(X, Y, q_n = 0.10) {
  X <- as.matrix(X)
  Y <- as.numeric(Y)

  n <- nrow(X)

  if (ncol(X) != 1L) {
    stop("This Gaussian-null study expects exactly one predictor column.")
  }
  if (length(Y) != n) {
    stop("Y must have the same number of observations as rows of X.")
  }

  x_mean <- mean(X[, 1L])
  y_mean <- mean(Y)

  x_centered <- X[, 1L] - x_mean
  y_centered <- Y - y_mean

  sxx <- sum(x_centered^2)
  if (sxx <= 0) {
    stop("X has zero sample variance.")
  }

  tau_hat <- sum(x_centered * y_centered) / sxx
  b_hat <- y_mean - tau_hat * x_mean
  residuals_hat <- Y - b_hat - tau_hat * X[, 1L]

  var_x_n <- sxx / n
  V_hat <- mean(x_centered^2 * residuals_hat^2) / var_x_n^2
}

```

```

gamma_hat <- sqrt(n) * tau_hat / sqrt(V_hat)
threshold_gamma <- qnorm(1 - q_n / 2)

list(
  score = gamma_hat,
  selected = abs(gamma_hat) >= threshold_gamma,
  threshold = threshold_gamma
)
}

```

5 One Gaussian-null data set

```

run_one_null_simulation <- function(seed, n, q) {
  set.seed(seed)

  X <- rnorm(n)
  Y <- rnorm(n)

  screening_test_ML(
    X = matrix(X, ncol = 1L),
    Y = Y,
    q_n = q
  )$selected
}

```

6 Estimate the FPR for one sample size

Each seed is explicit. The result is reproducible regardless of the number of processor cores and the scheduling of parallel tasks.

```

estimate_fpr_at_n <- function(n, B, q, seed_offset, mc.cores = 1L) {
  seeds <- as.integer(seed_offset + seq_len(B))

  one_draw <- function(seed) {
    run_one_null_simulation(seed = seed, n = n, q = q)
  }

  selected <- if (.Platform$OS.type == "windows" || mc.cores <= 1L) {
    unlist(lapply(seeds, one_draw), use.names = FALSE)
  } else {
    unlist(
      parallel::mclapply(
        X = seeds,
        FUN = one_draw,
        mc.cores = mc.cores,
        mc.set.seed = FALSE,
        mc.preschedule = FALSE
      ),

```

```

    use.names = FALSE
  )
}

fpr_hat <- mean(selected)

list(
  n = n,
  B = B,
  FPR_hat = fpr_hat,
  MC_SE = sqrt(fpr_hat * (1 - fpr_hat) / B)
)
}

```

7 Monte Carlo study

A different seed block is allocated to each sample size. This avoids overlap between the random-number streams used at different n .

```

seed_offsets <- cumsum(c(base_seed, head(B_by_n, -1L) + 1000L))

fpr_results <- Map(
  f = function(n, B, seed_offset) {
    estimate_fpr_at_n(
      n = n,
      B = B,
      q = q,
      seed_offset = seed_offset,
      mc.cores = n_cores
    )
  },
  n = n_values,
  B = B_by_n,
  seed_offset = seed_offsets
)

fpr_rate_summary <- do.call(
  rbind,
  lapply(fpr_results, as.data.frame)
)

fpr_rate_summary$FPR_minus_q <- fpr_rate_summary$FPR_hat - q
fpr_rate_summary$abs_FPR_minus_q <- abs(fpr_rate_summary$FPR_minus_q)
fpr_rate_summary$sqrt_n_times_abs_FPR_minus_q <-
  sqrt(fpr_rate_summary$n) * fpr_rate_summary$abs_FPR_minus_q
fpr_rate_summary$signal_to_MC_noise <-
  fpr_rate_summary$abs_FPR_minus_q / fpr_rate_summary$MC_SE

```

8 FPR convergence-rate table

```
knitr::kable(
  fpr_rate_summary,
  digits = 6,
  caption = paste0(
    "Gaussian-null FPR study: q = ", q,
    "; the number of null columns B_n increases with n"
  )
)
```

Table 1: Gaussian-null FPR study: $q = 0.5$; the number of null columns B_n increases with n

n	B	FPR_hat	MC_SE	FPR_minus_abs	FPR_minus_sqrt_n_times_abs_FPR_signal_to_MC_noise		
450	5000	0.501600	0.007071	0.001600	0.001600	0.033941	0.226275
675	7500	0.491867	0.005773	-0.008133	0.008133	0.211310	1.408921
1012	11245	0.506803	0.004715	0.006803	0.006803	0.216417	1.442952
1519	16878	0.495675	0.003849	-0.004325	0.004325	0.168570	1.123851
2278	25312	0.502765	0.003143	0.002765	0.002765	0.131992	0.879977
3417	37967	0.498933	0.002566	-0.001067	0.001067	0.062355	0.415703
5126	56956	0.504249	0.002095	0.004249	0.004249	0.304204	2.028109
7689	85434	0.502353	0.001711	0.002353	0.002353	0.206300	1.375357

9 Log-log rate estimate

The regression uses points whose estimated FPR deviation is at least twice the corresponding Monte Carlo standard error. This prevents Monte Carlo noise from dominating the fitted slope.

```
usable <- with(
  fpr_rate_summary,
  abs_FPR_minus_q > 2 * MC_SE
)

if (sum(usable) >= 3L) {
  rate_fit <- lm(
    log(abs_FPR_minus_q) ~ log(n),
    data = fpr_rate_summary,
    subset = usable
  )

  rate_fit_summary <- data.frame(
    n_points_used = sum(usable),
    estimated_log_log_slope = unname(coef(rate_fit)[2L]),
    target_slope = -0.5
  )

  knitr::kable(
    rate_fit_summary,
```

```

    digits = 4,
    caption = "Estimated log-log slope of the FPR deviation"
  )
} else {
  rate_fit <- NULL
  message(
    "Too few usable points. Increase B_by_n or reduce mc_error_fraction."
  )
}
}

```

```
## Too few usable points. Increase B_by_n or reduce mc_error_fraction.
```

10 Log-log plot

The dashed reference line has slope $-1/2$. It is anchored at the first usable point.

```

positive <- fpr_rate_summary$abs_FPR_minus_q > 0

plot(
  fpr_rate_summary$n[positive],
  fpr_rate_summary$abs_FPR_minus_q[positive],
  log = "xy",
  type = "b",
  xlab = "Sample size n",
  ylab = expression(abs(hat(FPR)[n] - q)),
  main = "Gaussian-null FPR convergence"
)

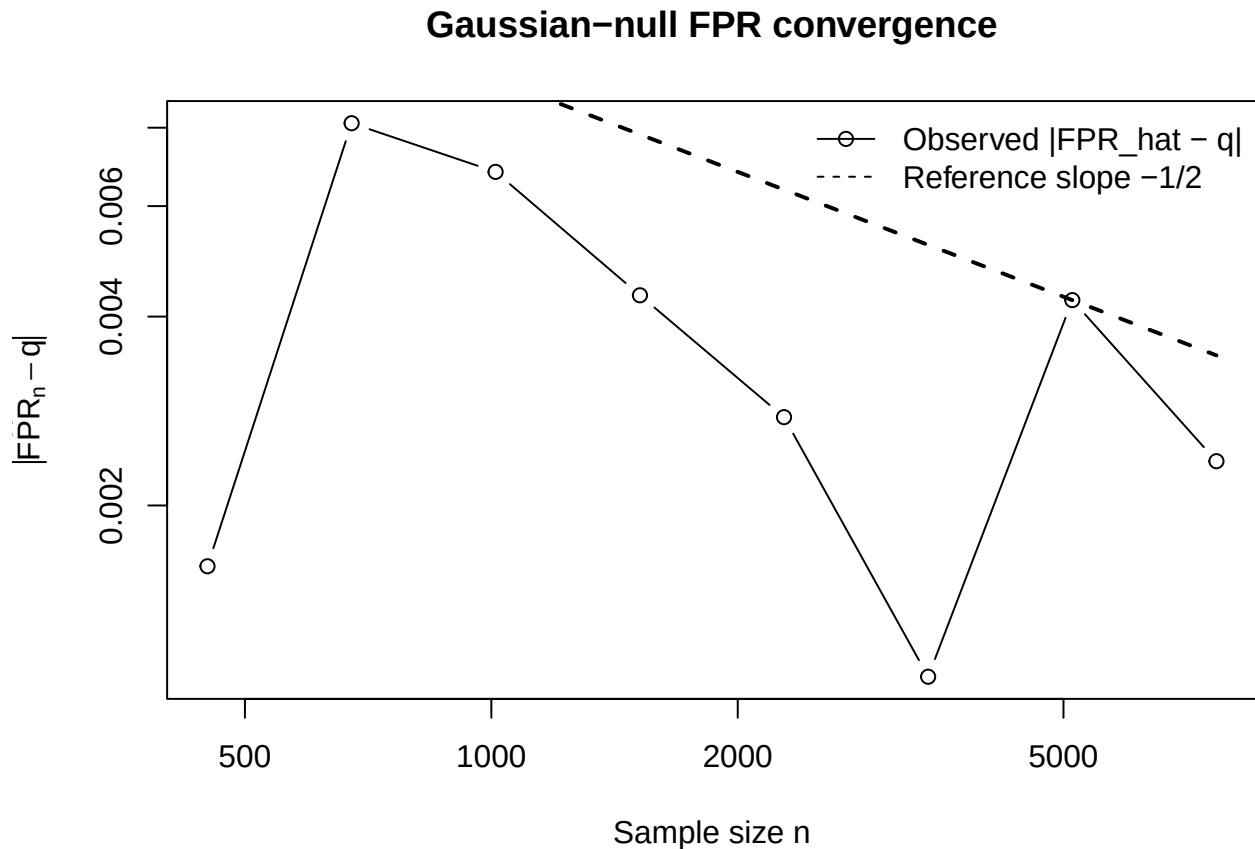
if (any(usable)) {
  anchor <- which(usable)[1L]

  reference_n <- range(fpr_rate_summary$n)
  reference_error <-
    fpr_rate_summary$abs_FPR_minus_q[anchor] *
    (reference_n / fpr_rate_summary$n[anchor])^(-0.5)

  lines(reference_n, reference_error, lty = 2, lwd = 2)

  legend(
    "topright",
    legend = c("Observed |FPR_hat - q|", "Reference slope -1/2"),
    lty = c(1, 2),
    pch = c(1, NA),
    bty = "n"
  )
}

```



11 Interpretation

- FPR_hat should be close to the nominal level q .
- A $C/\sqrt{n}$ bias suggests that `sqrt_n_times_abs_FPR_minus_q` remains approximately stable.
- The fitted log-log slope should be near -0.5 only over points where the FPR deviation is larger than Monte Carlo uncertainty.
- The value $q = 0.50$ is used here because it gives a more visible rejection probability; the same code works for $q = 0.10$.

12 Save results

```
saveRDS(
  list(
    parameters = list(
      q = q,
      n_values = n_values,
      B_by_n = B_by_n,
      mc_error_fraction = mc_error_fraction
    ),
    fpr_rate_summary = fpr_rate_summary
  ),
  file = "gaussian_null_fpr_convergence_results.rds"
```

```
)  
  
write.csv(  
  fpr_rate_summary,  
  file = "gaussian_null_fpr_convergence_summary.csv",  
  row.names = FALSE  
)
```